

Artificial Intelligence

Assignment 6

Group: Ritchie

1. Eshwar Reddy Poreddy: 221100642

(eshwaraa07@uni-koblenz.de)

2. Prajna Shetty: 222100479

(prajnashetty73@uni-koblenz.de)

3. Vamshidhar Pratap: 222202552

(vamshidharpratap@uni-koblenz.de)

4. Yangsun Kim: 220202400

(ykim@uni-koblenz.de)

1 Answer Set Programming

(10 Points)

Let P be the following extended logic program:

$$\begin{aligned} P : \quad & a \leftarrow \text{not } c. \\ & c \leftarrow b, \text{not } q. \\ & c \leftarrow \text{not } d, \text{not } e. \\ & e \leftarrow a, \text{not } d. \\ & d \leftarrow q. \\ & b. \end{aligned}$$

- Compute the reduct P^S of state $S = \{a, b\}$. Justify why S is not an answer set for P .
- Find all answer sets for P . Always mention the state S , its reduct P^S and justify why S is an answer set.
- Transform the program into an equivalent default theory $T = (W, \Delta)$ such that there is a one to one correspondence between extensions and answer sets.

a. $P^S = \{ a., c \leftarrow b., c., e \leftarrow a., d \leftarrow q., b. \}$

Minimal model of P^S is : $\{ a, b, c, d, e \}$ which is $\neq S$.
As S is not equal , it is not an answer set for P

b. $S1 = \{ b, c, d \}$

Reduct of $s1$, $P^{s1} = \{ c \leftarrow b., d \leftarrow q., b. \}$
Minimal model of P^{s1} is $\{c,d,b\} = S1$

c. $W = \{ a,b,c,d,e,q \}$

$$\Delta = \{ \delta 1 = \frac{:\sim c}{a},$$

$$\delta 2 = \frac{T:\sim c}{a},$$

$$\delta 3 = \frac{b:\sim q}{c},$$

$$\delta 4 = \frac{T:\sim d,\sim e}{c},$$

$$\delta 5 = \frac{a:\sim d}{e},$$

$$\delta 6 = \frac{q:T}{d},$$

$$\delta 7 = \frac{T:T}{b}, \}$$

2 Default Reasoning in Potassco (10 Points)

1. Write a constraint stating a literal and its contrary cannot both hold at the same time.

`:- holds(L, T), holds(neg(L), T).`

2. Write a rule stating the closed world hypothesis: if a literal holds at a point in time and we cannot prove that its contrary literal holds at the next point in time, then the literal continues to hold.

`holds(L, T+1) :- holds(L, T), not holds(neg(L), T+1), time(T), time(T+1).`

3. Write rules for a predicate `possible/2` that describes whether it is possible to perform each action at a point in time. This corresponds to the C set in STRIPS.

`possible(load, T) :- holds(walking, T), not holds(loaded, T).`

`possible(shoot, T) :- holds(loaded, T), holds(walking, T), not holds(dead, T).`

`-performed(A, T) :- action(A), time(T), not possible(A, T).`

4. Of course, we do not perform an action if we cannot prove that it is possible to perform it:

`-performed(A,T) :- action(A), time(T), not possible(A,T).`

5. Write rules describing the effects of performing each action.

`holds(loaded, T+1) :- performed(load, T), time(T), time(T+1).`

`holds(dead, T+1) :- performed(shoot, T), time(T), time(T+1).`

6. Execute the program. How can you see from the output what will happen?

clingo version 5.7.0

Reading from stdin

-.:15:23-41: info: atom does not occur in any rule head:

performed(load,T)

-.:16:21-40: info: atom does not occur in any rule head:

performed(shoot,T)

Solving...

Answer: 1

holds(walking,0) holds(neg(loaded),0) holds(neg(dead),0) possible(load,0) holds(neg(dead),1)
holds(neg(loaded),1) holds(walking,1) holds(walking,2) holds(neg(loaded),2) holds(neg(dead),2)
holds(neg(dead),3) holds(neg(loaded),3) holds(walking,3) holds(walking,4) holds(neg(loaded),4)
holds(neg(dead),4) holds(neg(dead),5) holds(neg(loaded),5) holds(walking,5) holds(walking,6)
holds(neg(loaded),6) holds(neg(dead),6) holds(neg(dead),7) holds(neg(loaded),7)
holds(walking,7) holds(walking,8) holds(neg(loaded),8) holds(neg(dead),8) holds(neg(dead),9)
holds(neg(loaded),9) holds(walking,9) holds(walking,10) holds(neg(loaded),10)
holds(neg(dead),10) holds(neg(dead),11) holds(neg(loaded),11) holds(walking,11)
holds(walking,12) holds(neg(loaded),12) holds(neg(dead),12) holds(neg(dead),13)
holds(neg(loaded),13) holds(walking,13) holds(walking,14) holds(neg(loaded),14)
holds(neg(dead),14) holds(neg(dead),15) holds(neg(loaded),15) holds(walking,15)
holds(walking,16) holds(neg(loaded),16) holds(neg(dead),16) holds(neg(dead),17)
holds(neg(loaded),17) holds(walking,17) holds(walking,18) holds(neg(loaded),18)
holds(neg(dead),18) holds(neg(dead),19) holds(neg(loaded),19) holds(walking,19)
holds(walking,20) holds(neg(loaded),20) holds(neg(dead),20) holds(neg(dead),21)
holds(neg(loaded),21) holds(walking,21) holds(walking,22) holds(neg(loaded),22)
holds(neg(dead),22) holds(neg(dead),23) holds(neg(loaded),23) holds(walking,23)
holds(walking,24) holds(neg(loaded),24) holds(neg(dead),24) holds(neg(dead),25)
holds(neg(loaded),25) holds(walking,25) holds(walking,26) holds(neg(loaded),26)
holds(neg(dead),26) holds(neg(dead),27) holds(neg(loaded),27) holds(walking,27)
holds(walking,28) holds(neg(loaded),28) holds(neg(dead),28) holds(neg(dead),29)
holds(neg(loaded),29) holds(walking,29) holds(walking,30) holds(neg(loaded),30)
holds(neg(dead),30) holds(neg(dead),31) holds(neg(loaded),31) holds(walking,31)
holds(walking,32) holds(neg(loaded),32) holds(neg(dead),32) holds(neg(dead),33)
holds(neg(loaded),33) holds(walking,33) holds(walking,34) holds(neg(loaded),34)
holds(neg(dead),34) holds(neg(dead),35) holds(neg(loaded),35) holds(walking,35)

[illegible]

holds(neg(dead),94) holds(neg(dead),95) holds(neg(loaded),95) holds(walking,95)
holds(walking,96) holds(neg(loaded),96) holds(neg(dead),96) holds(neg(dead),97)
holds(neg(loaded),97) holds(walking,97) holds(walking,98) holds(neg(loaded),98)
holds(neg(dead),98) holds(neg(dead),99) holds(neg(loaded),99) holds(walking,99)
possible(load,1) possible(load,2) possible(load,3) possible(load,4) possible(load,5)
possible(load,6) possible(load,7) possible(load,8) possible(load,9) possible(load,10)
possible(load,11) possible(load,12) possible(load,13) possible(load,14) possible(load,15)
possible(load,16) possible(load,17) possible(load,18) possible(load,19) possible(load,20)
possible(load,21) possible(load,22) possible(load,23) possible(load,24) possible(load,25)
possible(load,26) possible(load,27) possible(load,28) possible(load,29) possible(load,30)
possible(load,31) possible(load,32) possible(load,33) possible(load,34) possible(load,35)
possible(load,36) possible(load,37) possible(load,38) possible(load,39) possible(load,40)
possible(load,41) possible(load,42) possible(load,43) possible(load,44) possible(load,45)
possible(load,46) possible(load,47) possible(load,48) possible(load,49) possible(load,50)
possible(load,51) possible(load,52) possible(load,53) possible(load,54) possible(load,55)
possible(load,56) possible(load,57) possible(load,58) possible(load,59) possible(load,60)
possible(load,61) possible(load,62) possible(load,63) possible(load,64) possible(load,65)
possible(load,66) possible(load,67) possible(load,68) possible(load,69) possible(load,70)
possible(load,71) possible(load,72) possible(load,73) possible(load,74) possible(load,75)
possible(load,76) possible(load,77) possible(load,78) possible(load,79) possible(load,80)
possible(load,81) possible(load,82) possible(load,83) possible(load,84) possible(load,85)
possible(load,86) possible(load,87) possible(load,88) possible(load,89) possible(load,90)
possible(load,91) possible(load,92) possible(load,93) possible(load,94) possible(load,95)
possible(load,96) possible(load,97) possible(load,98) possible(load,99)

SATISFIABLE

Models : 1

Calls : 1

Time : 0.052s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)

CPU Time : 0.000s